TEMA 04

# Manejo de arrays

Desarrollo Web en Entorno Servidor
Tema 4 · Curso 2026/2027

Conceptos · Código · Ejemplos · Actividades

# Qué aprenderás

- Crear y modificar arrays con claves numéricas o de texto.
- Representar tablas de datos mediante arrays anidados.
- Recorrer listas, registros y matrices con foreach.
- Elegir la ordenación adecuada y conservar las claves cuando sea necesario.
- Consultar elementos, extraer claves y valores y generar enteros aleatorios.

Requisitos: variables, condiciones y bucles. Los ejemplos usan [] y PHP 8.x.

# Un array relaciona claves y valores

- En PHP un array es un mapa ordenado: cada clave identifica un valor.
- Las claves pueden ser enteros o cadenas; no se exige que sean consecutivas.
- Un valor puede ser otro array. Así se construyen estructuras anidadas.
- «Indexado» y «asociativo» describen usos habituales del mismo tipo array.

Usaremos claves sencillas y consistentes. Algunas claves se convierten de tipo: por ejemplo, "8" se convierte en el entero 8.

Referencia [1] · enlaces al final

# Una lista con índices numéricos

```php
$lenguajes = ["PHP", "Python", "Java"];
echo $lenguajes[0];       // PHP
$lenguajes[1] = "Go";
$lenguajes[] = "C#";
echo count($lenguajes);   // 4
```

- En esta lista los índices iniciales son 0, 1 y 2. [] añade un elemento.

En general, [] usa la siguiente clave entera según las claves del array; no asumas que siempre coincide con count().

Referencia [1] · enlaces al final

# Un registro con claves de texto

```php
$producto = [
    "nombre" => "Teclado",
    "precio" => 25.0,
    "stock" => 8,
];
echo $producto["nombre"]; // Teclado
$producto["stock"] = 7;
$producto["activo"] = true;
```

Las claves de texto se escriben entre comillas. Una misma clave solo conserva un valor: asignarla otra vez lo reemplaza.

Referencia [1] · enlaces al final

# Claves ausentes y valores null

```php
$usuario = ["nombre" => "Ana", "telefono" => null];

var_dump(isset($usuario["telefono"])); // false
var_dump(array_key_exists("telefono", $usuario));
// true: la clave existe, aunque su valor sea null

$ciudad = $usuario["ciudad"] ?? "Sin indicar";
echo $ciudad; // Sin indicar
```

isset exige que la clave exista y su valor no sea null. ?? usa la alternativa cuando falta la clave o su valor es null.

Referencia [2] · enlaces al final

# Eliminar y volver a numerar

```php
$colores = ["rojo", "verde", "azul"];
unset($colores[1]);
// Quedan las claves 0 y 2.

$lista = array_values($colores);
// $lista: [0 => "rojo", 1 => "azul"]
echo count($colores); // 2
```

unset no renumera. count cuenta elementos; no indica la mayor clave. Para recorrer un array con huecos, usa foreach.

Referencia [1, 3] · enlaces al final

# Claves y valores por separado

```php
$notas = ["Ana" => 8, "Luis" => 6];
$nombres = array_keys($notas);
$valores = array_values($notas);

print_r($nombres); // [0 => Ana, 1 => Luis]
print_r($valores); // [0 => 8, 1 => 6]
```

- Los comentarios resumen el contenido; print_r añade su propio formato.

Ambas funciones devuelven un array nuevo. Si necesitas clave y valor juntos, foreach es más directo.

Referencia [3] · enlaces al final

# Recorrer un array asociativo

```php
$notas = ["Ana" => 8, "Luis" => 6, "Eva" => 9];
foreach ($notas as $nombre => $nota) {
    echo $nombre . ": " . $nota . "<br>";
}
```

- Cada vuelta obtiene una pareja clave-valor.
- Se mantiene el orden actual del array.

Los nombres son fijos en este ejemplo. Para valores externos mostrados en HTML, utiliza escape HTML.

Referencia [4] · enlaces al final

# Una matriz de dos dimensiones

```
$matriz = [
    [10, 20, 30],
    [40, 50, 60],
];
echo $matriz[1][2]; // 60
$matriz[0][1] = 25;
```

- Primer índice: fila. Segundo índice: elemento de esa fila.
- La matriz del ejemplo tiene 2 filas y 3 columnas.

PHP permite que cada fila tenga un tamaño distinto. Aquí usamos filas del mismo tamaño para representar una tabla.

# Una lista de registros

```
$productos = [
    ["nombre" => "Teclado", "precio" => 25.0],
    ["nombre" => "Ratón", "precio" => 15.0],
];
echo $productos[0]["nombre"]; // Teclado
$productos[1]["precio"] = 18.0;
$productos[] = ["nombre" => "Monitor", "precio" => 150.0];
```

El array exterior es una lista; cada elemento interior es un registro con claves conocidas.

# Registros identificados por una clave

```php
$alumnos = [
    "A01" => ["nombre" => "Ana", "nota" => 8],
    "A02" => ["nombre" => "Luis", "nota" => 6],
];
echo $alumnos["A02"]["nombre"]; // Luis
$alumnos["A01"]["nota"] = 9;
```

- A01 y A02 son identificadores, no posiciones de una lista.

Elige la clave según el problema. Un identificador estable ayuda a localizar un registro sin buscar por su posición.

# Recorrer una matriz

```
$matriz = [[10, 20, 30], [40, 50, 60]];
foreach ($matriz as $fila) {
    foreach ($fila as $valor) {
        echo $valor . " ";
    }
    echo "<br>";
}
```

- En el navegador se ven dos líneas: 10 20 30 y 40 50 60.

El bucle exterior selecciona una fila; el interior recorre sus elementos.

# Recorrer registros conocidos

```php
$productos = [
    ["nombre" => "Teclado", "precio" => 25.0],
    ["nombre" => "Ratón", "precio" => 15.0],
];
foreach ($productos as $producto) {
    echo $producto["nombre"] . ": ";
    echo $producto["precio"] . " EUR<br>";
}
```

Si conoces los campos que necesitas, no hace falta un segundo foreach. Accede a nombre y precio de forma explícita.

# Una tabla HTML con datos del array

```php
$notas = ["Ana" => 8, "Luis" => 6];
echo "<table><tr><th>Alumno</th><th>Nota</th></tr>";
foreach ($notas as $nombre => $nota) {
    $seguro = htmlspecialchars(
        $nombre, ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8"
    );
    echo "<tr><td>" . $seguro . "</td>";
    echo "<td>" . $nota . "</td></tr>";
}
echo "</table>";
```

Los nombres se escapan para mostrarlos como texto HTML. Las notas de este ejemplo son números; el diseño visual corresponde a CSS.

Referencia [5] · enlaces al final

# Qué ordena cada función

| Función | Orden ascendente / descendente | Claves |
|---|---|---|
| sort / rsort | Por valores | Renumera desde 0 |
| asort / arsort | Por valores | Conserva la asociación |
| ksort / krsort | Por claves | Conserva la asociación |

Estas funciones modifican el array recibido. No devuelven un array ordenado para asignarlo a otra variable.

Referencia [6] · enlaces al final

# sort y rsort: ordenar una lista

```php
$precios = [30, 10, 20];
sort($precios, SORT_NUMERIC);
// $precios: [10, 20, 30]

rsort($precios, SORT_NUMERIC);
// $precios: [30, 20, 10]
```

- Para números usamos SORT_NUMERIC. Para texto, puede usarse SORT_STRING.

No escribas $precios = sort($precios): perderías el array y guardarías el valor de retorno de la función.

Referencia [6] · enlaces al final

# asort y arsort: conservar las claves

```php
$notas = ["Ana" => 8, "Luis" => 6, "Eva" => 9];
asort($notas, SORT_NUMERIC);
// Orden: Luis => 6, Ana => 8, Eva => 9

arsort($notas, SORT_NUMERIC);
// Orden: Eva => 9, Ana => 8, Luis => 6
```

- La nota sigue vinculada al nombre correcto después de ordenar.

Con sort se perderían los nombres y las claves pasarían a ser 0, 1 y 2.

Referencia [6] · enlaces al final

# ksort y krsort: ordenar por clave

```php
$notas = ["Luis" => 6, "Ana" => 8, "Eva" => 9];
ksort($notas, SORT_STRING);
// Orden: Ana => 8, Eva => 9, Luis => 6

krsort($notas, SORT_STRING);
// Orden: Luis => 6, Eva => 9, Ana => 8
```

Se ordenan los nombres, no las notas. La ordenación de texto con SORT_STRING no equivale a una ordenación lingüística española.

Referencia [6] · enlaces al final

# La ordenación cambia el significado

| Operación sobre ["B" => 20, "A" => 10] | Resultado |
|---|---|
| sort(..., SORT_NUMERIC) | [0 => 10, 1 => 20] |
| asort(..., SORT_NUMERIC) | ["A" => 10, "B" => 20] |
| krsort(..., SORT_STRING) | ["B" => 20, "A" => 10] |

Cada fila parte del array original. Antes de ordenar, decide si la clave identifica algo que debes conservar.

Referencia [6] · enlaces al final

# Buscar sin confundir tipos

```php
$codigos = [10, 20, 30];
var_dump(in_array(20, $codigos, true));   // true
var_dump(in_array("20", $codigos, true)); // false

$posicion = array_search(10, $codigos, true);
if ($posicion !== false) {
    echo $posicion; // 0: es una posición válida
}
```

El tercer argumento true activa la comparación estricta. array_search devuelve una clave o false; no confundas la clave 0 con false.

Referencia [7] · enlaces al final

# Números aleatorios

```php
$dado = random_int(1, 6);
$notas = [];
for ($i = 0; $i < 5; $i++) {
    $notas[] = random_int(0, 10);
}
print_r($notas);
```

- Los dos extremos están incluidos. Los valores pueden repetirse.

random_int genera enteros uniformes y seguros criptográficamente. mt_rand sigue existiendo, pero no debe usarse para fines de seguridad.

Referencia [8] · enlaces al final

ACTIVIDAD

# Práctica: inventario de una tienda

- Crea una lista de cuatro productos con nombre, precio y stock.
- Muestra en una tabla HTML solo los productos con stock mayor que 0.
- Calcula el valor del inventario: suma precio × stock de cada producto.
- Crea un array asociativo nombre → precio y ordénalo de mayor a menor precio conservando los nombres.
- Comprueba un inventario vacío y uno donde todos los productos tengan stock 0.

Supuestos: nombres únicos, precios no negativos y stock entero no negativo. Usa condiciones y bucles; todavía no necesitas funciones propias.

# Repaso y errores frecuentes

- Un array anidado se accede por niveles: $datos[0]["nombre"].
- foreach recorre también arrays con huecos en sus claves numéricas.
- sort renumera; asort conserva claves; ksort ordena por claves.
- Una clave ausente y una clave con valor null no son lo mismo.
- Comprueba los casos vacío, un solo elemento, duplicados y claves que no existen.

Ampliación posterior: ordenar registros completos con usort requiere conocer funciones de comparación.

# Referencias y alcance

[1] Arrays: estructura y claves

[2] array_key_exists e isset

[3] array_keys y array_values

[4] Recorridos con foreach

[5] Escape HTML

[6] Ordenación de arrays

[7] Búsqueda estricta

[8] random_int y mt_rand

Adaptación del material original del docente. Revisión: septiembre de 2026. Ejemplos para PHP 8.x; ampliaciones señaladas.